

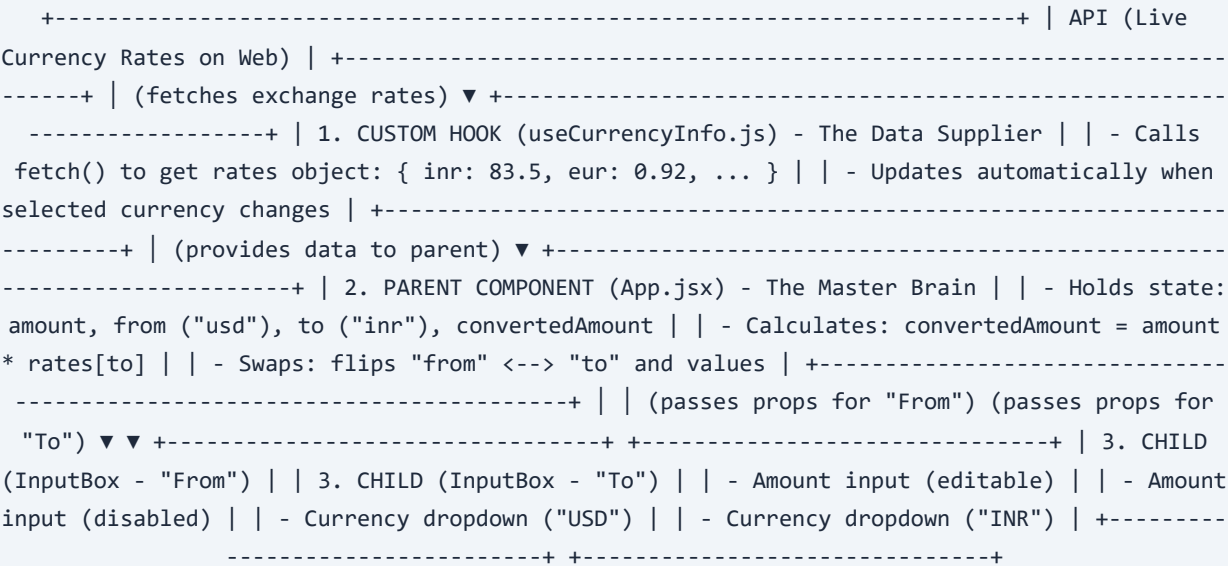
REACT MASTERY & ARCHITECTURE NOTES

Currency Converter Project - Deep Dive Guide

A beginner-friendly, crystal-clear explanation of Custom Hooks, Component Reusability, and State Orchestration.

1. The Big Picture: Architecture & Why We Split Code

When you build an application in React, you should never write all your code in one giant file. Instead, we divide responsibilities into three distinct layers:



File	Type	Primary Role
useCurrencyInfo.js	Custom Hook (Function)	Fetches API data and manages rates state. Doesn't care about HTML/UI.
InputBox.jsx	Reusable UI Component	Renders a single input card. Used twice (for "From" and "To").
App.jsx	Parent Component	Holds the core state, does the conversion math, and ties everything together.

2. Deep Dive: useCurrencyInfo.js (Custom Hook)

Why was this created instead of writing fetch inside App.jsx?

- **Separation of Concerns:** UI components should only handle user interaction. Data-fetching logic belongs in a hook.
- **Reusability:** If you build a dashboard or header tomorrow that also needs currency rates, you can call `useCurrencyInfo('usd')` in 1 line without rewriting `fetch` and `useEffect`.
- **Clean Code:** It keeps `App.jsx` short, readable, and easy to maintain.

The Complete Code:

```
import { useEffect, useState } from "react";

function useCurrencyInfo(currency) {
  // 1. State to store the fetched currency rates object
  const [data, setData] = useState({});

  // 2. useEffect runs whenever the 'currency' argument changes
  useEffect(() => {
    fetch(`https://cdn.jsdelivr.net/npm/@fawazahmed0/currency-api@latest/v1/currency
      .then((res) => res.json()) // Convert stream to JSON
      .then((res) => setData(res[currency])) // Save only the rates dictionary
    }, [currency]); // Dependency array

    // 3. Return the rates data
    return data;
  }

  export default useCurrencyInfo;
```

💡 **Why is it `setData(res[currency])` and NOT `setData(res)` ?**

The API response looks like this:

```
{
  "date": "2026-09-12",
  "usd": {
    "inr": 83.5,
    "eur": 0.92,
    "jpy": 145.2
  }
}
```

Notice that the exchange rates are inside `res["usd"]` (or `res[currency]`). If you wrote `setData(res)`, your state would contain `"date"` and `"usd"` instead of currency rates, breaking both the dropdown and math!

3. Deep Dive: `InputBox.jsx` (Reusable Component)

Why was this component created?

Look at your currency converter screen:

- The **"From" card** has an amount input box and a currency selector.
- The **"To" card** has an amount input box and a currency selector.

They look identical! In standard HTML, you would copy-paste 40 lines of HTML twice. In React, you build **ONE reusable template** (`InputBox`) and use it twice by passing different **props**.

Props Explained in Detail:

Prop Name	Default	What It Does
<code>label</code>	-	Displays text like "From" or "To" above the input.
<code>amount</code>	-	The number currently displayed in the number input.
<code>onAmountChange</code>	-	Function called when the user types a new number. Tells the parent to update state.
<code>onCurrencyChange</code>	-	Function called when the user selects a new currency in the dropdown.
<code>currencyOptions</code>	<code>[]</code>	Array of currency strings: <code>["usd", "inr", "eur", ...]</code> used to generate options.
<code>selectedCurrency</code>	<code>"usd"</code>	The currency currently active in the dropdown.
<code>amountDisable</code>	<code>false</code>	If <code>true</code> , locks the amount input so user cannot type (used for the "To" result box).
<code>currencyDisable</code>	<code>false</code>	If <code>true</code> , locks the dropdown.

Important Techniques Inside `InputBox.jsx` :

1. What is `useId()` doing?

```
const amountInputId = useId();  
...  
<label htmlFor={amountInputId}>{label}</label>  
<input id={amountInputId} ... />
```

`useId()` generates a unique string (e.g. `:r1:`, `:r2:`) guaranteed to be unique across renders. It connects the `<label>` with the `<input>` for accessibility and allows clicking the label to automatically focus the input.

2. Safe Function Invocation: `onAmountChange && onAmountChange( ... )`

```
onChange={(e) => onAmountChange && onAmountChange(Number(e.target.value))}
```

The `&&` check prevents app crashes if the parent forgot to pass an `onAmountChange` prop. Also, `Number(e.target.value)` converts the raw text input into a true numeric value.

3. Why use `key={currency}` in the loop?

```
{currencyOptions.map((currency) => (  
  <option key={currency} value={currency}>  
    {currency}  
  </option>  
))}
```

React's Virtual DOM requires a unique `key` on repeated items. It allows React to identify which items have changed, added, or removed without re-rendering the entire list.

4. Deep Dive: `App.jsx` (The Orchestrator)

`App.jsx` is the parent component that coordinates the entire app. It controls 4 states and 2 operations:

The State Variables:

```
const [amount, setAmount] = useState(0);           // Input amount to convert (e
const [from, setFrom] = useState("usd");           // Source currency
const [to, setTo] = useState("inr");               // Destination currency
const [convertedAmount, setConvertedAmount] = useState(0); // Output result
```

Hook Integration & Options Generation:

```
// 1. Call our custom hook with the base currency:
const currencyInfo = useCurrencyInfo(from);

// 2. Extract keys from the rates object into an array:
const currencyOptions = Object.keys(currencyInfo);
// Result: ["inr", "eur", "gbp", "cad", "aud", ...]
```

The Two Main Actions:

1. The Swap Logic:

```
const swap = () => {
  setFrom(to);           // Currency "from" takes the value of "to"
  setTo(from);           // Currency "to" takes the value of "from"
  setAmount(convertedAmount); // Amount gets the converted value
  setConvertedAmount(amount); // Converted amount gets original amount
};
```

2. The Conversion Calculation:

```
const convert = () => {
  // Math: Multiply user amount by destination currency rate:
  // e.g., 10 * currencyInfo["inr"] (which is 83.5) = 835
  setConvertedAmount(amount * currencyInfo[to]);
};
```

How `App.jsx` renders both `InputBoxes`:

```

<form onSubmit={(e) => { e.preventDefault(); convert(); }}>

  {/* FROM BOX */}
  <InputBox
    label="From"
    amount={amount}
    onAmountChange={setAmount}
    onCurrencyChange={setFrom}
    currencyOptions={currencyOptions}
    selectedCurrency={from}
  />

  {/* SWAP BUTTON */}
  <button type="button" onClick={swap}>swap</button>

  {/* TO BOX */}
  <InputBox
    label="To"
    amount={convertedAmount}
    currencyOptions={currencyOptions}
    onCurrencyChange={setTo}
    selectedCurrency={to}
    amountDisable /* Read only! User cannot type output amount */
  />

  <button type="submit">
    Convert {from.toUpperCase()} to {to.toUpperCase()}
  </button>
</form>

```

5. Step-by-Step Execution Lifecycle

1. **Initial Render:** App loads. `from` is `"usd"`. `useCurrencyInfo("usd")` runs its `fetch()`.
2. **API Response:** Data arrives. `setData(res["usd"])` updates the hook's state.
3. **Re-render:** `currencyOptions` now holds hundreds of currencies. Both dropdowns populate with options.
4. **User Input:** User enters `50` in the "From" input. `onAmountChange` fires → `setAmount(50)` updates `amount`.
5. **User Converts:** User clicks "Convert USD to INR" → Form runs `convert()` → `setConvertedAmount(50 * 83.5)` → "To" box displays `4175` !